

Pivoting in the Solution of Linear Systems of Equations

Why choosing the right row matters

Outline

- 1 Motivation
- 2 Pivoting Strategies
- 3 Numerical Example
- 4 Conclusion

Solving Linear Systems: Gaussian Elimination

- A linear system $A\mathbf{x} = \mathbf{b}$ is commonly solved using **Gaussian elimination**.
- The method transforms A into an upper triangular matrix using row operations based on **pivot elements**.
- The pivot is the diagonal entry used to eliminate the entries below it.

a_{kk} (the pivot at step k)

- **Problem:** what happens if $a_{kk} = 0$, or is very small?

The Problem Without Pivoting

- If the pivot $a_{kk} = 0$: division by zero $\Rightarrow$ the algorithm fails.
- If a_{kk} is very small (close to zero): the multiplier $m_{ik} = a_{ik}/a_{kk}$ becomes very large.
- Large multipliers amplify **rounding errors**, causing numerical instability.
- Conclusion: the naive algorithm is not robust, even when a mathematical solution exists.

Partial Pivoting

- At step k , search column k (from row k downward) for the entry with the **largest absolute value**.
- Swap that row with row k before eliminating.

$$\text{choose } i^* = \arg \max_{i \geq k} |a_{ik}|, \quad \text{swap rows } k \leftrightarrow i^*$$

- Guarantees all multipliers satisfy $|m_{ik}| \leq 1$.
- Standard approach used in most numerical libraries (e.g. LAPACK, NumPy, SciPy).

Complete (Total) Pivoting

- Searches for the largest absolute value in the **entire remaining submatrix**, not just the column.
- Requires swapping both rows *and* columns.
- More numerically stable, but computationally expensive ($O(n^3)$ extra comparisons).
- Rarely used in practice: partial pivoting is usually stable enough.

Example System

Consider the linear system:

$$\begin{array}{l} 0x_1 + 1x_2 = 1 \\ 1x_1 + 2x_2 = 3 \end{array} \quad A = \begin{pmatrix} 0 & 1 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$$

Without pivoting: $a_{11} = 0$, elimination fails immediately (division by zero).

With partial pivoting: swap $R_1 \leftrightarrow R_2$ since $|a_{21}| = 1 > |a_{11}| = 0$.

Example: Solving After Pivoting

After swapping rows:

$$\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$$

The system is already upper triangular. Back-substitution gives:

$$x_2 = 1, \quad x_1 = 3 - 2(1) = 1$$

Solution: $x_1 = 1, x_2 = 1$.

Python Example: Gaussian Elimination with Partial Pivoting

```
1 import numpy as np
2
3 def gauss_pivot(A, b):
4     A = A.astype(float).copy()
5     b = b.astype(float).copy()
6     n = len(b)
7
8     for k in range(n - 1):
9         # Partial pivoting
10        i_max = np.argmax(
11            np.abs(A[k:, k])
12        ) + k
13
14        if i_max != k:
15            A[[k, i_max]] = \
16            A[[i_max, k]]
17            b[[k, i_max]] = \
18            b[[i_max, k]]
```

```
1 # Elimination
2 for i in range(k + 1, n):
3     m = A[i, k] / A[k, k]
4     A[i, k:] -= m * A[k, k:]
5     b[i] -= m * b[k]
6
7 # Back-substitution
8 x = np.zeros(n)
9
10 for i in range(n - 1, -1, -1):
11     x[i] = (
12         b[i] - A[i, i+1:] @ x[i+1:]
13     ) / A[i, i]
14
15 return x
```

Python Example: Verification with NumPy

- NumPy's built-in solver already implements pivoting internally (via LAPACK routines).

```
import numpy as np

A = np.array([[0, 1], [1, 2]])
b = np.array([1, 3])

x = np.linalg.solve(A, b)
print(x)      # -> [1.  1.]
```

- Same result as our manual implementation: pivoting made the zero-pivot case solvable.

- Naive Gaussian elimination can fail or become unstable when pivots are zero or small.
- **Partial pivoting** swaps rows to place the largest available value on the diagonal.
- This keeps multipliers bounded and greatly improves numerical stability.
- Used by default in professional numerical libraries (NumPy, SciPy, MATLAB, LAPACK).
- **Rule of thumb:** always pivot in practice it is cheap insurance against numerical disaster.

Thank you